RESUME

PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

Oleh:

NADHEA DENI PUTRI

120140128

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI, PRODUKSI DAN INDUSTRI

INSTITUT TEKNOLOGI SUMATERA

LAMPUNG SELATAN

2023

BAB I
DASAR PEMROGRAMAN PYTHON

A. Sintaks Dasar

a. Statement

Semua perintah yang bisa dieksekusi Python disebut statement. Pada Python akhir dari sebuah statement adalah baris baru (newline) tapi dimungkinkan membuat statement yang terdiri dari beberapa baris menggunakan backslash (\).

b. Baris dan Indentasi

Python tidak menggunakan kurung kurawal sebagai grouping blok kode melainkan menggunakan spasi ataupun tab (4 spasi). kode yang berada diblok yang sama harus memiliki jumlah spasi yang sama di awal.

B. Variabel dan Tipe Data Primitif

Variabel merupakan lokasi penyimpanan yang berguna untuk menyimpan suatu data atau suatu nilai. Dalam mendeklarasikan suatu variabel dalam pemrograman, perlu diketahui tipe-tipe data yang berhubungan dengan variabel yang akan dideklarasikan, di bawah ini merupakan tipe data yang ada :

Tipe Data	Jenis	Nilai
bool	Boolean	True atau false
int	Bilangan bulat	Seluruh bilangan bulat
float	Bilangan real	Seluruh bilangan real
string	Teks	Kumpulan karakter

C. Operator

Python memiliki sejumlah operator, yaitu :

a. Operator Aritmatika

Operator aritmatika adalah operator yang digunakan untuk melakukan operasi matematika, seperti penjumlahan, pengurangan, perkalian, pembagian, dan sebagainya. Tabel berikut menunjukkan jenis operator aritmatika:

Operator	Nama dan Fungsi	Contoh
+	Penjumlahan, menjumlahkan 2 buah operand	x + y
-	Pengurangan, mengurangkan 2 buah operand	x – y
*	Perkalian, mengalikan 2 buah operand	x * y
/	Pembagian, membagi 2 buah operand	x / y
**	Pemangkatan, memangkatkan bilangan	x **y
//	Pembagian bulat, menghasilkan hasil bagi tanpa koma	x // y
%	Modulus, menghasilkan sisa pembagian 2 bilangan	x % y

b. Operator Perbandingan

Operator perbandingan adalah operator yang digunakan untuk membandingkan 2 buah nilai. Hasil perbandingannya adalah True atau False tergantung kondisi.

Operator	Nama dan Fungsi	Contoh
>	Lebih besar dari – Hasilnya True jika nilai sebelah kiri lebih besar dari nilai sebelah kanan	x > y
<	Lebih kecil dari – Hasilnya True jika nilai sebelah kiri lebih kecil dari nilai sebelah kanan	x < y
==	Sama dengan – Hasilnya True jika nilai sebelah kiri sama dengan nilai sebelah kanan	x == y
!=	Tidak sama dengan – Hasilnya True jika nilai sebelah kiri tidak sama dengan nilai sebelah kanan	x != y
>=	Lebih besar atau sama dengan – Hasilnya True jika nilai sebelah kiri lebih besar atau sama dengan nilai sebelah kanan	x >= y
<=	Lebih kecil atau sama dengan – Hasilnya True jika nilai sebelah kiri lebih kecil atau sama dengan nilai sebelah kanan	x <= y

c. Operator Penugasan

Operator penugasan adalah operator yang digunakan untuk memberi nilai ke variabel. a = 7 adalah contoh operator penugasan yang memberi nilai 7 di kanan ke variabel a yang ada di kiri.

Operator	**Penjelasan**	**Contoh**
=	Menugaskan nilai yang ada di kanan ke operand di sebelah kiri	c = a + b menugaskan a + b ke c
+=	Menambahkan operand yang di kanan dengan operand yang ada di kiri dan hasilnya ditugaskan ke operand yang di kiri	c += a sama dengan c = c + a
-=	Mengurangi operand yang di kanan dengan operand yang ada di kiri dan hasilnya ditugaskan ke operand yang di kiri	c -= a sama dengan c = c - a
*=	Mengalikan operand yang di kanan dengan operand yang ada di kiri dan hasilnya ditugaskan ke operand yang di kiri	c *= a sama dengan c = c * a
/=	Membagi operand yang di kanan dengan operand yang ada di kiri dan hasilnya ditugaskan ke operand yang di kiri	c /= a sama dengan c = c * a

**=	Memangkatkan operand yang di kanan dengan operand yang ada di kiri dan hasilnya ditugaskan ke operand yang di kiri	c **= a sama dengan c = c ** a
//=	Melakukan pembagian bulat operand di kanan terhadap operand di kiri dan hasilnya disimpan di operand yang di kiri	c //= a sama dengan c = c // a
%=	Melakukan operasi sisa bagi operand di kanan dengan operand di kiri dan hasilnya disimpan di operand yang di kiri	c %= a sama dengan c = c % a

d. Operator Logika

Operator logika adalah operator yang digunakan untuk melakukan operasi logika.

Operator	Penjelasan	Contoh
and	Hasilnya adalah True jika kedua operandnya bernilai benar	x and y
or	Hasilnya adalah True jika salah satu atau kedua operandnya bernilai benar	x or y
not	Hasilnya adalah True jika operandnya bernilai salah (kebalikan nilai)	not x

e. Operator Bitwise

Operator bitwise adalah operator yang melakukan operasi bit terhadap operand. Operator ini beroperasi bit per bit sesuai dengan namanya. Sebagai misal, angka 2 dalam bit ditulis 10 dalam notasi biner dan angka 7 ditulis 111. Pada tabel di bawah ini, misalkan x = 10 (0000 1010) dalam biner dan y = 4 (0000 0100) dalam biner.

Operator	Nama	Contoh
&	Bitwise AND	x& y = 0 (0000 0000)
|	Bitwise OR	x | y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)
^	Bitwise XOR	x ^ y = 14 (0000 1110)
>>	Bitwise right shift	x>> 2 = 2 (0 000 0010)
<<	Bitwise left shift	x<< 2 = 40 (0010 1000)

f. Operator Identitas

Operator identitas adalah operator yang memeriksa apakah dua buah nilai (atau variabel) berada pada lokasi memori yang sama.

Operator	Penjelasan	Contoh
is	True jika kedua operand identik (menunjuk ke objek yang sama)	x is True
is not	True jika kedua operand tidak identik (tidak merujuk ke objek yang sama)	x is not True

g. Operator Keanggotaan

Operator keanggotaan adalah operator yang digunakan untuk memeriksa apakah suatu nilai atau variabel merupakan anggota atau ditemukan di dalam suatu data (string, list, tuple, set, dan dictionary).

Operator	Penjelasan	Contoh
in	True jika nilai/variabel ditemukan di dalam data	5 in x
not in	True jika nilai/variabel tidak ada di dalam data	5 not in x

D. Tipe Data Bentukan

Ada 4, dengan perbedaan penggunaan:

- List

  Sebuah kumpulan data yang terurut, dapat diubah, dan memungkinkan ada anggota yang sama
- Tuple

  Sebuah kumpulan data yang terurut, tidak dapat diubah, dan memungkinkan ada anggota yang sama
- Set

  Sebuah kumpulan data yang tidak berurutan, tidak terindeks, dan tidak memungkinkan ada anggota yang sama
- Dictionary

  Sebuah kumpulan data yang tidak berurutan, dapat diubah, tidak memungkinkan ada anggota yang sama

Tipe data dasar	Contoh nilai	Penjelasan
List	[1, 2, 3, 4, 5] atau [‘apple’, ‘banana’, ‘cherry’] atau [‘xyz’, 768, 2.23]	Data untaian yang menyimpan berbagai tipe data dan isinya bisa diubah-ubah
Tuple	(‘xyz’, 1, 3.14)	Data untaian yang menyimpan berbagai tipe data tapi isinya tidak bisa diubah
Dictionary	{‘firstName’:‘Joko’, ‘lastName’: ‘Widodo’}	Data untaian yang menyimpan berbagai tipe data berupa pasangan penunjuk dan nilai
Set	{ ‘apple’, ‘banana’, ‘cherry’ }	Data untaian yang menyimpan berbagai tipe data dan elemen datanya harus unik

E. Percabangan

Dalam bahasa pemrograman Python, terdapat beberapa percabangan yaitu IF, IF-ELSE, dan IF-ELSE-IF.

a. Percabangan IF

Sebagai contoh, akan dibuat suatu program dengan bahasa Python untuk menentukan bilangan negatif.

Kode di atas hanya bisa menentukan bilangan negatif saja, tanpa menentukan bilangan lainnya yaitu positif dan nol. Jika pengguna memasukkan angka positif atau nol, program tidak mengeluarkan keluaran apapun

b. Percabangan IF-ELSE

Untuk mengeluarkan output ketika user memasukkan selain angka negatif, perlu ditambahkan percabangan lagi. Kita dapat mengimplementasikan percabangan IF-ELSE.

Penambahan line 8-9 akan mengeluarkan keluaran untuk bilangan positif. Namun, kode di atas tidak dapat mengeluarkan keluaran bila pengguna memasukkan bilangan nol. Agar program bisa menentukan bilangan nol, kita dapat menggunakan percabangan IF-ELSE-IF.

c. Percabangan IF-ELSE-IF

Kita dapat menggunakan percabangan IF-ELSE-IF untuk melengkapi program sebelumnya.

d. Nested IF

Selain menggunakan code seperti gambar diatas untuk menentukan bilangan positif, negatif atau bilangan nol, kita dapat mengubah format code menjadi percabangan bersarang (nested if) dengan menempatkan percabangan di dalam percabangan. Kode program dapat dilihat seperti gambar dibawah ini.

F. Perulangan

Dalam python terdapat dua jenis perulangan , yaitu perulangan for dan perulangan while. Dalam implementasi nya perulangan digunakkan jika ingin mengulang sesuatu sebanyak n kali.

a. Perulangan For

Pada perulangan for biasa digunakkan untuk iterasi pada urutan berupa list, tuple, atau string. Sintaks dasar pada perulangan for di python:

- Perulangan for pada list

  output :

- Perulangan for pada string

  output:

- Perulangan for pada tuple

  output :

Untuk menggunakkan perulangan for dapat juga dilakukan dengan menggunakkan indeks atau range. Dimana range akan mengembalikan nilai dari 0 kemudian bertambah 1 sampai batas nilai yang ditentukan.

Sintaks umum penggunaan range pada for :

1. Menggunakkan satu parameter

keterangan : Perulangan akan dilakukan dari indeks 0 sampai kurang dari x.

2. Menggunakkan 2 parameter

keterangan : Perulangan akan dilakukan dari indeks ke-x sampai kurang dari y.

3. Menggunakkan 3 parameter

keterangan : Perulangan akan dilakukan dari indeks ke-x sampai kurang dari y dengan indeks bertambah/increment sejumlah z.

contoh (mengakses tuple dengan menggunakkan indeks) :

output :

Dalam for juga dikenal penggunaan break dan continue. Statement break digunakkan ketika ingin keluar dari baris perulangan sedangkan continue digunakkan ketika ingin melewati kondisi tertentu dan melanjutkan ke baris program selanjutnya.

contoh penggunaan break (mencari bunga “melati” pada tuple bunga):

output:

contoh penggunaan continue (melewati spasi pada kalimat “Teknik Informatika”):

output :

b. Perulangan While

Dengan menggunakkan while maka dapat dilakukan perulangan selama kondisi tertentu terpenuhi. sintaks umum pada perulangan

while :

contoh (menghitung karakter c sebelum tanda !) :

output :

G. Fungsi

Dengan menggunakkan fungsi kita dapat mengeksekusi suatu blok kode tanpa harus menulisnya berulang-ulang. Contoh:

Hasil:

Dalam fungsi juga dapat digunakan parameter default yang dapat dilihat hasilnya saat pemanggilan fungsi dengan parameter tidak lengkap:

Hasil:

Fungsi di Python juga memungkinkan pendeteksian banyak parameter secara otomatis melalui parameter spesial *args (arguments) dan **kwargs (keyworded arguments). Dalam *args, parameter fungsi dianggap sebagai satu kesatuan list, sedangkan dalam **kwargs parameter fungsi dianggap sebagai satu kesatuan dictionary.

Contoh penggunaan *args:

Contoh penggunaan **kwargs:

BAB II
OBJEK DAN KELAS DALAM PYTHON
(KONSTRUKTOR, SETTER, DAN GETTER)

A. Kelas

Kelas atau class pada python bisa kita katakan sebagai sebuah blueprint (cetakan) dari objek (atau instance) yang ingin kita buat. Dengan menggunakan kelas kita dapat mendesain objek secara bebas. Kelas berisi dan mendefinisikan atribut/properti dan metode untuk objeknya nanti. Satu buah kelas dapat membuat banyak objek dan kelas sendiri tidak bisa langsung digunakan, harus diimplementasikan menjadi sebuah objek dulu, dapat disebut Instansiasi. contohnya: kelas Mobil, kelas Manusia, dan kelas Kucing.

Untuk membuat kelas, gunakan kata kunci class diikuti oleh nama kelas tersebut dan tanda titik dua. Contoh:

Dapat dilihat diatas terdapat __init__(), method tersebut adalah konstruktor untuk membuat kelas mobil. Untuk method __init__() lebih lanjut akan dibahas pada pembahasan selanjutnya. Kelas juga dapat dibuat kosong dengan menggunakan kata kunci pass seperti contoh berikut ini:

Kelas mungkin merupakan cara termudah bagi kita untuk mengorganisasikan data ketika melakukan komputasi ilmiah. Meskipun demikian hal ini bukan tanpa kelemahan, terkadang dengan mengorganisasikan data kita ke dalam kelas, program yang kita buat menjadi lambat ketika dijalankan. Seperti yang sudah dijelaskan diatas bahwa di dalam sebuah kelas terdapat variabel (atribut/properti) dan fungsi (method).

a. Atribut/Property

Dalam suatu kelas, umumnya dideklarasikan sebuah variabel yang akan disebut sebagai sebuah atribut. Karena dideklarasikan di dalam sebuah kelas, maka setiap objek yang dibentuk dari kelas tersebut juga akan memiliki atribut yang dimiliki oleh kelas tersebut. Terdapat 2 jenis atribut, yaitu atribut kelas dan atribut objek. Atribut kelas merupakan sifat yang dimiliki oleh sebuah kelas dan juga akan dimiliki oleh setiap objek. Sedangkan atribut objek adalah sebuah atribut dari masing - masing objek.

Contoh :

Output :

Dari contoh diatas dapat kita lihat bahwa jumlah_tangan dan jumlah_kaki merupakan atribut kelas yang nilainya akan sama dalam setiap objek yang dibuat menggunakan kelas tersebut. Sedangkan nama, pekerjaan, dan tahun_lahir merupakan atribut objek yang akan memiliki nilai dari masing - masing objek.

Untuk memanggil atribut dapat dengan syntax :

b. Method

Selain atribut, elemen lain yang terdapat dalam suatu kelas adalah method. Method adalah suatu fungsi yang terdapat di dalam kelas. Sama halnya seperti atribut, semua objek yang dibuat menggunakan kelas yang sama akan memiliki method yang sama pula. Method dapat diibaratkan sebagai sebuah aktivitas/proses yang dapat dilakukan oleh sebuah objek. Misalkan objek manusia dapat makan, berjalan, mencuci_piring.

Contoh :

Output :

Dicontohkan bahwa kelas Orang memiliki method berjalan_kedepan() yang akan mencetak "Melangkah" dan "Posisi berpindah". Untuk memanggil method mirip seperti memanggil atribut, namun dengan tambahan tanda kurung "()" seperti berikut:

B. Objek

Objek adalah sesuatu yang "mewakili" kelas. Objek disini berfungsi sebagai pengganti pemanggilan sebuah kelas, maka sebuah objek hanya dapat mewakili sebuah kelas saja. Untuk membuat sebuah objek yang mewakili sebuah kelas, kita dapat membuatnya dengan cara memanggil nama dari kelas yang diinginkan, ditambah dengan tanda kurung ().

Contoh : Diibaratkan kita memiliki sebuah kelas bernama "kelas_yang_dipanggil", maka untuk membuat objek bernama "ini_objek" cukup seperti ini :

Pembuatan objek mempermudah pemanggilan kelas, terutama ketika nama dari suatu kelas terlalu panjang dan kelas tersebut kadang perlu berinteraksi dengan kelas lain.

C. Magic Method

Magic method adalah metode yang diawali dan diakhiri dengan double underscore (dunder). Method ini tidak dipanggil secara langsung, tapi dipanggil sistem secara internal ketika melakukan sesuatu seperti menggunakan operator tambah (__add__), membuat objek (__init__), dan lain-lain.

Tujuan utama dari penggunaan magic method adalah untuk mengubah sifat (behavior) bawaan dari suatu objek. Untuk melihat apa saja magic method bawaan python pada class int, gunakan sintaks dir(int).

Magic method tidak terbatas pada kelas int saja, tetapi ada di setiap jenis objek dan variabel. Salah satu contoh magic method yaitu __add__(). Method ini ditambahkan agar user dapat melakukan operasi penambahan (+) secara langsung pada objek, tanpa mengakses atribut isi objek (dalam hal ini yaitu self.angka). Tanpa penambahan magic method __add__(), maka statement print(N + P) akan menghasilkan error.

output:

D. Konstruktor

Konstruktor adalah method yang "pasti" dijalankan secara otomatis pada saat sebuah objek dibuat untuk mewakili kelas tersebut. Seperti dengan method lain, konstruktor dapat melakukan operasi seperti melakukan print. Selain operasi method dasar,

konstruktor dapat menerima argumen yang diberikan ketika objek dibuat. Argumen ini akan diproses di dalam kelas nantinya.

Contoh :

Output :

Argumen juga bisa dimasukkan menggunakan input pengguna.

E. Destruktor

Destruktor adalah fungsi yang dipanggil ketika user menghapus objek. Fungsi ini bekerja secara otomatis, jadi tidak perlu dilakukan pemanggilan. Tujuan adanya destruktor adalah melakukan final cleaning up atau “bersih-bersih” terakhir sebelum sebuah objek benar-benar dihapus dari memori.

output:

F. Setter dan Getter

Setter dan getter digunakan untuk melakukan enkapsulasi agar tidak terjadi perubahan data secara tidak sengaja. Setter adalah method yang digunakan untuk menetapkan nilai suatu atribut khususnya atribut private dan protected (akan diajarkan lebih jelas pada materi minggu selanjutnya), sedangkan getter digunakan untuk mengambil nilai.

Terlihat dari contoh di atas, terdapat 2 cara untuk mengakses atribut protected, yaitu menggunakan getter dan memanggil nama atribut secara langsung. Berbeda dengan atribut protected, atribut private tidak dapat diakses secara langsung tanpa menggunakan getter seperti contoh di bawah.

G. Decorator

Selain menggunakan fungsi setter dan getter tambahan seperti pada contoh sebelumnya, dalam Python kita juga dapat memanfaatkan property decorator untuk mendapatkan hasil serupa. Bedanya, melalui property decorator ini kita tidak perlumembuat fungsi lagi dengan nama yang berbeda-beda (cukup menggunakan 1 buah nama/variabel).

BAB III
ABSTRAKSI DAN ENKAPSULASI
(VISIBILITAS FUNGSI DAN VARIABEL, RELASI ANTAR KELAS)

A. Abstraksi

Abstraksi adalah konsep OOP dimana model yang dibuat hanya memperlihatkan atribut yang esensial dan menyembunyikan detail-detail yang tidak penting dari user. gunanya untuk mengurangi kompleksitas.

User mengetahui apa yang objek lakukan, tapi tidak tau mekanisme yang terjadi di belakang layar bagaimana. Contohnya ketika mengendarai mobil, user mengetahui bagaimana menyalakan mobil, menjalankan, menghentikan, dll, tetapi tidak mengetahui mekanisme apa yang terjadi pada mobil ketika mendapat perintah di atas.

Ketika mendefinisikan kelas, sebenarnya sedang membuat Abstrak dari suatu Objek. Kelas merupakan bentuk abstrak atau cetak biru (blue print) dari suatu objek nyata. Wujud nyata suatu kelas dinamakan instance (Objek). Contoh : Str adalah kelas, sedangkan teks “Python” adalah objek.

B. Enkapsulasi (Encapsulation)

Enkapsulasi adalah sebuah metode yang digunakan untuk mengatur struktur kelas dengan cara menyembunyikan alur kerja dari kelas tersebut. Yang dimaksud dengan struktur kelas tersebut adalah property dan method. Dengan konsep ini, kita dapat “menyembunyikan” property dan method dari suatu kelas agar hanya property dan method tertentu saja yang

dapat diakses dari luar kelas. Dengan menghalangi akses dari luar kelas tersebut, elemen penting yang terdapat dalam kelas dapat lebih terjaga, dan menghindari kesalahan jika elemen tersebut diubah secara tidak sengaja.

Abstraksi adalah cara untuk menyembunyikan informasi yang tidak dibutuhkan, sedangkan enkapsulasi adalah cara menyembunyikan atribut suatu entitas serta metode untuk melindungi informasi dari luar. Untuk membatasi hak akses terhadap property dan method dalam suatu kelas, terdapat 3 jenis access modifier yang terdapat dalam python, yaitu public access, protected access, dan private access.

- Public Access Modifier

  Pada umumnya ketika kita mendeklarasikan suatu variabel atau method, maka itulah public access modifier. Setiap class, variable dan method yang dibuat secara default merupakan public.

- Protected Access Modifier

  Jika suatu variabel dan method dideklarasikan secara protected, maka variable dan method tersebut hanya dapat diakses oleh kelas turunan darinya. Cara mendeklarasikannya dengan menambahkan 1 underscore (_) sebelum variable atau method.

● Private Access Modifier

Jika suatu variabel dan method dideklarasikan secara private, maka variable dan method tersebut hanya dapat diakses di dalam kelas itu sendiri, private access merupakan yang paling aman. Dalam mendeklarasikannya, hanya perlu menambahkan double underscore ( ) sebelum nama variable dan methodnya.

● Setter dan Getter

Seperti yang sudah dijelaskan pada minggu yang lalu, setter adalah sebuah method yang digunakan untuk mengatur sebuah property yang ada di dalam suatu kelas/objek. Sedangkan getter adalah sebuah method yang digunakan untuk mengambil nilai dari suatu property. Dikarenakan property dengan access modifier private hanya dapat diakses dari dalam kelas tersebut, dengan metode inilah kita dapat mengaksesnya dari luar kelas. Terdapat 2 cara untuk membuat setter dan getter, yaitu dengan tanpa decorator dan dengan decorator.

Perlu diketahui bahwa penggunaan setter getter dengan decorator hanya dapat dilakukan terhadap properti dengan access modifier protected dan private. Jika dilakukan terhadap properti yang bersifat public (dengan nama yang sama), maka akan mengakibatkan error

C. Object

a. Membuat Instance Objects

Disini terdapat suatu kelas mahasiswa seperti berikut :

Untuk membuat instance dari kelas yang telah dibuat dapat dilakukan dengan menggunakan nama dari class kemudian argumen diterima oleh metode init.

b. Mengakses Atribut Objek

Dari objek yang telah dibuat dapat dilakukan pengaksesan atribut dari objek dengan menggunakkan operator dot(titik)

c. Menambah, menghapus, dan Mengubah atribut object

Objek yang sudah dibuat dapat dimodifikasi seperti ditambah, dihapus, ataupun diubah atributnya. Misal ingin mengubah semester dari mahasiswa 3, karena ternyata mahasiswa 3 sudah semester 6.

del merupakan fungsi yang menghapus atribut semester dari maha3. Sehingga pada output terlihat bahwa saat pemanggilan method tampil_profil kedua setelah atribut dihapus terjadi error.

Cara yang lebih baik untuk memodifikasi atribut dari suatu objek dapat menggunakkan fungsi berikut :

● getattr(obj, name[, default]) – Mengakses atribut dari objek

● hasattr(obj, name) – Memeriksa apakah suatu objek memiliki atribut tertentu

● setattr(obj, name, value) – Mengatur nilai atribut. Jika ternyata atribut tidak ada, maka atribut tersebut akan dibuat.

● delattr(obj, name) – Menghapus atribut dari suatu objek.

BAB IV
PEWARISAN DAN POLIMORFISME
(OVERLOADING, OVERRIDING, DYNAMIC CAST)

A. Inheritance (Pewarisan)

Inheritance adalah salah satu konsep dasar dari Pemrograman Berbasis Objek (OOP). Pada inheritance, kita dapat menurunkan kelas dari kelas lain untuk hirarki kelas yang saling berbagi atribut dan metode. Contohnya terdapat base class “Animal” dan terdapat class “Horse” yang merupakan turunan dari class “Animal”. ini berarti class “Horse” memiliki atribut dan method yang sama dengan class “Animal”. Dan objek “Horse” dapat menggantikan objek “Animal” dalam aplikasi.

● Inheritance Identik

Inheritance identik merupakan pewarisan yang menambahkan constructor pada class child sehingga class child memiliki constructornya sendiri tanpa menghilangkan constructor pada class parentnya. Metode ini ditandai dengan adanya class child yang menggunakan constructor dan menggunakan kata kunci super().

● Menambah karakteristik pada child class

Pada child class dapat ditambahkan beberapa fitur tambahan baik atribut maupun method sehingga child class tidak identik dengan parent class.

B. Polymorphism

Polymorphism berarti banyak (poly) dan bentuk (morphism), dalam Pemrograman Berbasis Objek konsep ini memungkinkan digunakannya suatu interface yang sama untuk memerintah objek agar melakukan aksi atau tindakan yang mungkin secara prinsip sama namun secara proses berbeda.

Polymorphism merupakan kemampuan suatu method untuk bekerja dengan lebih dari satu tipe argumen. Pada bahasa lain (khususnya C++), konsep ini sering disebut dengan method overloading. Pada dasarnya, Python tidak menangani hal ini secara khusus. Hal ini disebabkan karena Python merupakan suatu bahasa pemrograman yang bersifat duck typing(dynamic typing).

C. Override/Overriding

Pada konsep OOP di python kita dapat menimpa suatu metode yang ada pada parent class dengan mendefinisikan kembali method dengan nama yang sama pada child class. Dengan begitu maka method yang ada parent class tidak berlaku dan yang akan dijalankan adalah method yang terdapat di child class.

Pada contoh terlihat bahwa terdapat parent class yaitu class bangunDatar dengan method tampil. Kemudian terdapat child class yaitu class Persegi yang memiliki method yang sama dengan method pada parent class. Dengan begitu ketika melakukan instansiasi objek child class dan memanggil method tampil() maka yang akan dipanggil ialah method pada child class, method pada parent class tidak berlaku.

D. Overloading

Metode overloading mengizinkan sebuah class untuk memiliki sekumpulan fungsi dengan nama yang sama dan argumen yang berbeda. Akan tetapi, Python tidak mengizinkan pendeklarasian fungsi (baik pada class ataupun tidak) dengan nama yang sama. Hal itu menyebabkan implementasi overloading pada python menjadi “tricky”. Secara umum overloading memiliki beberapa signature, yaitu jumlah argumen, tipe argumen, tipe keluaran dan urutan argumen. Akan tetapi, seperti yang telah dijelaskan python tidak menangani overloading secara khusus. Karena python adalah bahasa pemrograman yang bersifat duck typing (dynamic typing), overloading secara tidak langsung dapat diterapkan.

E. Multiple Inheritance

Python mendukung pewarisan ke banyak kelas. Kelas dapat mewarisi dari banyak orang tua. Bentuk syntax multiple inheritance.

Kita juga dapat mewarisi dari kelas turunan atau disebut warisan bertingkat. Pewarisan ini dapat dilakukan hingga ke dalaman berapa pun. Dalam kelas turunan dari kelas dasar dan turunannya dapat diwarisi ke dalam kelas turunan yang baru

F. Method Resolution Order di Python

MRO adalah urutan pencarian metode dalam hirarki class. Hal ini terutama berguna dalam multiple inheritance. Urutan MRO dalam python yaitu bawah-atas dan kiri-kanan. Artinya, method dicari pertama kali di kelas objek. jika tidak ditemukan, pencarian berlanjut ke super class. Jika terdapat banyak superclass (multiple inheritance), pencarian dilakukan di kelas yang paling kiri dan dilanjutkan ke kelas sebelah kanan.

G. Dynamic Cast

Dynamic cast atau type conversion adalah proses mengubah nilai dari satu tipe data ke tipe data lainnya seperti dari string ke int atau sebaliknya. Ada 2 tipe konversi yaitu:

a. Implisit

Python secara otomatis mengkonversikan tipe data ke tipe data lainnya tanpa ada campur tangan pengguna.

b. Eksplisit

Pengguna mengubah tipe data sebuah objek ke tipe data lainnya dengan fungsi yangsudah ada dalam python seperti int(), float(), dan str(). dapat berisiko terjadinya kehilangan data.

H. Casting

a) Downcasting: Parent class mengakses atribut yang ada pada kelas bawah (child class)

b) Upcasting: Child class mengakses atribut yang ada pada kelas atas (parent class)

c) Type casting: Konversi tipe kelas agar memiliki sifat/perilaku tertentu yang secara default tidak dimiliki kelas tersebut. Karena Python merupakan bahasa pemrograman berorientasi objek, maka semua variabel atau instansi di Python pada dasarnya merupakan objek (kelas) yang sifat/perilakunya dapat dimanipulasi jika dibutuhkan (umumnya melalui magic method) Magic method tidak hanya terbatas pada contoh di atas namun terdapat juga banyak lainnya, beberapa magic method juga digunakan untuk konsep operator overloading seperti __add__ (pertambahan), __sub__ (pengurangan), __mul__ (perkalian), dll.

BAB V
KELAS ABSTRAK DAN INTERFACE
(KELAS ABC, METACLASS)

I. Konsep Abstraksi

Sejatinya tidak semua yang kita lihat dan buat adalah sebuah objek konkrit tetapi ada juga yang bersifat abstrak. Saat ini kita telah sering kali menggunakan objek konkrit seperti apel, pisang, lingkaran, dan lainnya. Abstraksi dalam konsep OOP adalah sebuah yang dibuat hanya memperlihatkan atribut yang esensial dan menyembunyikan detail-detail yang tidak penting dari user. Gunanya adalah untuk mengurangi kompleksitas. User dapat mengetahui apa yang objek lakukan, tapi tidak tau mekanisme yang terjadi di belakang layar bagaimana. Contohnya ketika mengendarai mobil, user engetahui bagaimana menyalakan mobil, menjalankan, menghentikan, dll, tetapi tidak mengetahui mekanisme apa yang terjadi pada mobil ketika mendapat perintah di atas.

A. Kelas Abstrak dalam Python

Kelas Abstrak adalah sebuah class yang tidak bisa diinstansiasi (tidak bisa dibuat menjadi objek) langsung dan berperan sebagai ‘kerangka dasar’ bagi class turunannya. Di dalam kelas abstrak umumnya akan memiliki sebuah abstract method. Abstract method adalah sebuah method yang harus diimplementasikan ulang di dalam class anak (child class). Abstract method ditulis tanpa isi dari method, melainkan hanya ‘signature’-nya saja. Signature dari sebuah method adalah bagian method yang terdiri dari nama method dan parameternya (jika ada).

Kelas Abstrak digunakan di dalam inheritance (pewarisan class) untuk ‘memaksakan’ atau OVERRIDE method yang sama bagi seluruh class yang diturunkan dari abstract class. Kelas Abstrak digunakan untuk membuat struktur logika penurunan di dalam pemrograman objek. Contoh pemakaian kelas abstrak misalnya dapat dipakai pada konsep mobil, dimana tiap-tiap mobil tentunya memiliki suatu kesamaan namun implementasi yang berbeda (misal dari segi kecepatan, bahan bakar, dll).

B. Implementasi Kelas Abstrak dengan Modul ABC

Python memiliki modul untuk menggunakan Abstract Base Classes (ABC). Fungsi abstrak pada kelas abstrak ditandai dengan memberikan decorator @abstractmethod pada fungsi yang dibuat.

Dari potongan kode diatas akan menghasilkan error jika kelas tersebut langsung dijadikan sebuah objek. Untuk mengatasi hal tersebut dan melakukan implementasi terhadap kelas abstrak diperlukan sebuah kelas ‘child’ dari kelas abstrak tersebut.

Membuat kelas ‘child’ untuk kelas abstrak sendiri ada ketentuannya, salah satunya adalah ketika membuat kelas ‘child’ dari kelas abstrak kita harus membuat kembali seluruh metode/fungsi yang ada pada parent class (kelas abstrak). Contohnya sebagai berikut:

Selain fungsi sederhana seperti gambar diatas, sebuah kelas abstrak juga bisa memiliki konstruktor dan fungsi biasa (yang tidak sepenuhnya kosong).

Misal, sama halnya ketika membuat konstruktor kelas pada umumnya, untuk membuat konstruktor pada kelas abstrak kita dapat menggunakan def __init__ seperti pada contoh berikut berikut:

Lalu, untuk implementasi fungsi biasa (konkret) pada kelas abstrak juga dapat dilakukan seperti contoh di bawah ini (yaitu fungsi get_warna).

II. Interface

A. Definisi Interface

● Interface adalah koleksi dari method atau fungsi-fungsi yang perlu disediakan oleh implementing class (child class).
● Interface mengandung metode yang bersifat abstract. Metode abstract akan memiliki satu-satunya deklarasi karena tidak adanya implementasi.
● Pengimplementasian sebuah interface adalah sebuah cara untuk menulis kode yang elegan dan terorganisir.

Perbedaan Interface dan Abstract class

Python interface	Python abstract class
Semua metode dari sebuah (formal) interface adalah abstrak	Sebuah abstract class dapat mempunyai metode abstract begitu juga dengan metode konkrit
Interface digunakan jika semua fitur perlu untuk diimplementasikan secara berbeda untuk objek yang berbeda	Abstract class digunakan ketika ada beberapa fitur umum yang dimiliki oleh semua objek
Interface cenderung lebih lambat jika dibandingkan dengan abstract class	Abstract class lebih cepat (tidak semua implementasi perlu ditulis pada child class)

B. Informal interface

Karena pengimplementasian interface di Python berbeda dengan bahasa lain seperti C++ dan Java, interface di Python kadang lebih diartikan sebagai sebuah konsep tanpa aturan yang terlalu ketat. Informal interface adalah kelas yang mendefinisikan metode atau fungsi-fungsi yang bisa di override/implementasi namun tanpa adanya unsur paksaan (cukup diimplementasi hanya bila dibutuhkan). Untuk mengimplementasikannya kita harus membuat kelas konkrit. Kelas konkrit adalah subclass dari (abstrak) interface yang menyediakan implementasi dari method atau fungsi-fungsi di kelas interface. Untuk melakukan implementasi menggunakan inheritance.

Pada contoh di atas, kebun binatang Ragunan dapat mengecek banyaknya hewan (__len__) dan mengecek keberadaan hewan (__contains__) pada kebun binatang tersebut. Namun, ketika dibutuhkan list dari hewan-hewan (melalui iterasi) yang berada pada kebun binatang Ragunan, maka akan menghasilkan error.

Untuk mengatasi error tersebut, pada kelas KebunBinatang perlu kita implementasikan sebuah fungsi baru yaitu __iter__ sehingga memungkinkan terjadinya iterasi untuk mendapatkan list hewan yang ada.

Dalam kasus ini, fungsi __iter__ pada kelas KebunBinatang perlu diimplementasikan karena dibutuhkan untuk mendapatkan/meng-iterasi list hewan yang ada. Namun, implementasi __iter__ ini tidak dipaksakan oleh kelas parent dari KebunBinatang yaitu kelas Hewan. Oleh karena itu, konsep ini disebut dengan informal interface. Hasil outputnya sekarang adalah sebagai berikut:

C. Formal Interface

Selain informal interface, terdapat pula formal interface. Pada formal interface kelas parent (abstrak) dapat dibangun hanya dengan sedikit baris kode, untuk kemudian diimplementasikan pada kelas turunannya (konkret). Implementasi ini bersifat wajib/dipaksakan sehingga dinamakan formal interface. Salah satu cara membuat formal interface adalah dengan abstract class method seperti pada contoh kelas abstrak sebelumnya.

3. Metaclass

A. Apa itu Metaclass?

Tidak seperti bahasa pemrograman lain seperti C++ dan Java yang memiliki suatu tipe dasar (misal int, float, dll), pada Python semua tipe pada dasarnya adalah suatu objek/kelas juga (termasuk int, float, dll).

Oleh karena itu, kita bisa saja membuat sebuah kelas turunan (subclass) dari int atau tipe dasar lainnya yang memiliki karakteristik tambahan seperti contoh berikut.

Konsep tipe ini bahkan juga berlaku pada objek dan kelas yang masih kosong seperti contoh di bawah.

Seperti yang terlihat pada output di atas, tipe dari objek mahasiswa adalah “Kosong” dan tipe dari kelas Kosong adalah “type”. Mengapa demikian? Di dalam Python, terdapat konsep hirarki dimana objek dibentuk dari kelas (class), sedangkan kelas dibentuk dari hirarki yang lebih tinggi yaitu metaclass (dalam hal ini berupa contoh metaclass tersebut adalah “type”).

Dengan kata lain, metaclass adalah tipe kelas spesial yang berfungsi untuk membuat kelas-kelas pada Python (atau biasa disebut juga dengan pabrik kelas/class factory).

B. Implementasi/Contoh Kasus Metaclass

1. Menggunakan type

Dibawah ini adalah contoh kasus pembuatan metaclass menggunakan type. Jika variabel “terpanggil” dipanggil, maka metaclass type tersebut akan membuat sebuah class bernama “dedikorbuzet”, dengan tuple berisi nama parent class (dikosongkan saja jika class yang dibuat tidak melakukan inheritance dari class manapun), dan atribut dari kelas tersebut yang berbentuk dictionary, dengan sintaks.

2. Menggunakan parameter metaclass

Selain dengan langsung menggunakan type untuk membuat sebuah kelas, kita juga dapat membuat sebuah kelas turunan hasil modifikasi terhadap type itu sendiri. Untuk memodifikasinya, kita perlu membuat sebuah kelas turunan dengan basis dari type

Seperti yang kita ketahui, bahwa sebuah type adalah metaclass. Maka, jika kita membuat sebuah kelas turunan dari type, maka kelas tersebut juga merupakan sebuah metaclass. Dari gambar diatas, kita membuat sebuah metaclass yang akan secara otomatis membuat atribut ‘jenis_metaclass’ dengan isi “IniJugaMeta metaclass”

Secara default, metaclass yang digunakan saat membuat sebuah kelas adalah type. Setelah membuat sebuah metaclass hasil turunan dari type, maka kita dapat membuat sebuah kelas dengan metaclass hasil modifikasi kita sendiri.